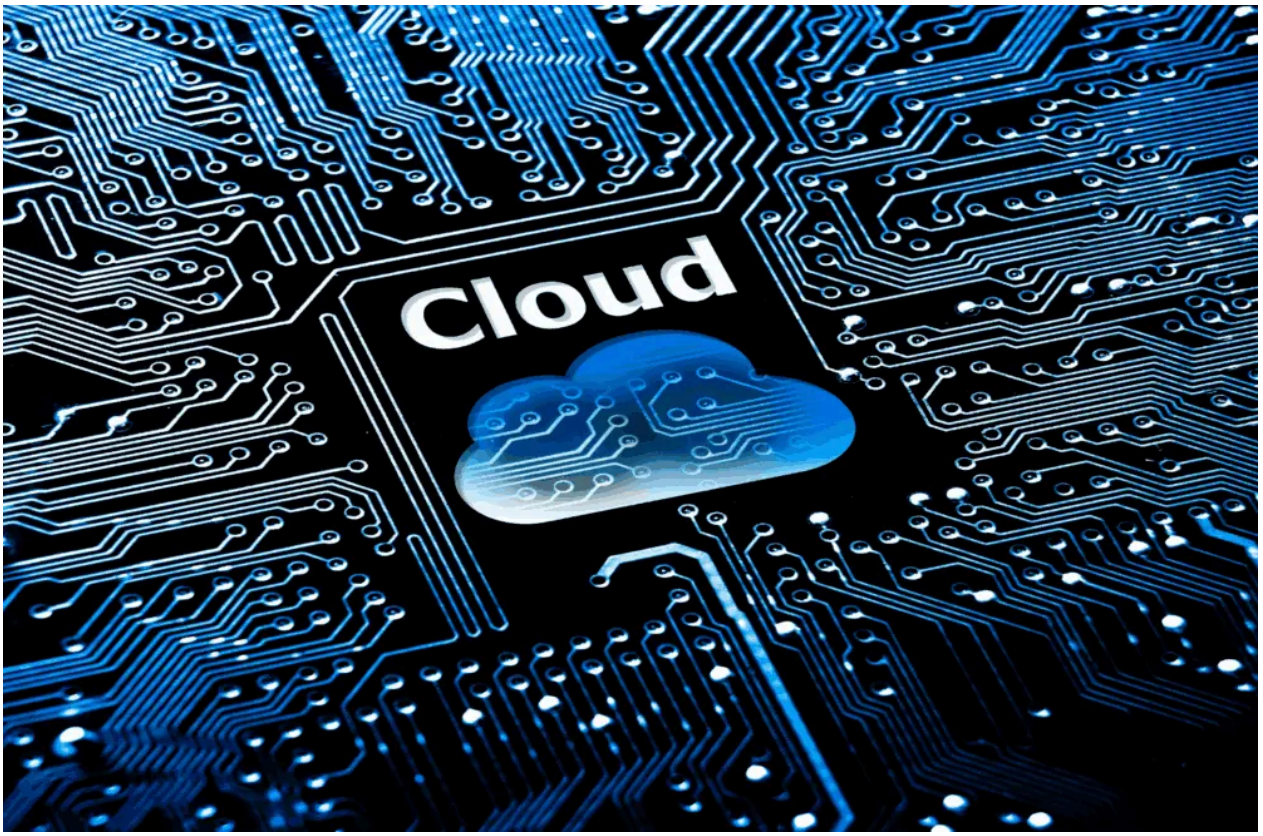
DESPLEGAMIENTO EN AWS Y AZURE DE INFRAESTRUCTURA CON TERRAFORM



Jaume Morte
Jose Tejero
Juan Miguel Aguilar

ÍNDICE

1. DESCRIPCIÓN DEL PROYECTO.....3
2. IMPLEMENTACIÓN DEL PROYECTO.....4
3. DIFICULTADES.....13
4. TRABAJO FUTURO.....15

1. DESCRIPCIÓN DEL PROYECTO

El objetivo de este proyecto es desplegar una infraestructura equivalente o complementaria con dos proveedores, que en este caso son AWS y Azure. Manteniendo así la coherencia en el código. En este caso desplegamos un wordpress.

2. IMPLEMENTACIÓN DEL PROYECTO

Primero de todo vamos a crear los archivos de configuración de terraform, vamos a dividir las configuraciones en varios archivos. En este caso lo dividiremos en aws y azure. Los archivos se llaman aws.tf y azure.tf. Seguidamente una vez creados, seguiremos con los archivos que nos indicarán las variables, los proveedores y los outputs. Que se llamaran variables.tf, provider.tf y outputs.tf.

El archivo de azure es un poco más complicado que el de aws ya que tendremos que crear nosotros las cosas, es decir que en aws ya nos vienen cosas por defecto, como por ejemplo las subredes, vpcs, etc.

Foto de aws.tf

aws.tf azure.tf variables.tf

aws.tf > resource "aws_instance" "awsvm1"

```
1  provider "aws" {
2    region = var.aws_region
3  }
4  resource "aws_security_group" "allow_ssh" {
5    name = "allow_ssh"
6    ingress {
7      from_port   = 22
8      to_port     = 22
9      protocol    = "tcp"
10     cidr_blocks = ["0.0.0.0/0"]
11   }
12 }
13
14 resource "aws_instance" "awsvm1" {
15   instance_type        = "t2.micro"
16   ami                  = "ami-0532be01f26a3de55"
17   iam_instance_profile = "LabInstanceProfile"
18   key_name             = "vockey"
19   vpc_security_group_ids = [aws_security_group.allow_ssh.id]
20
21   tags = {
22     Name = "awsvm1"
23   }
24 }
```

Foto de [azure.tf]

azure.tf > resource "azurerm_subnet" "subnet"

```
provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = "azurevm1_rg"
  location = var.azurerm_region
}

resource "azurerm_virtual_network" "vnet" {
  name                = "azurevm1_vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name                 = "azurevm1_subnet"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_public_ip" "publicip" {
  name                = "azurevm1-publicip"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Dynamic"
}

resource "azurerm_network_interface" "nic" {
  name                = "azurevm1_nic"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
```

```
  ip_configuration {
    name                          = "azurevm1_ipconfig"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.publicip.id
  }
}

resource "azurerm_network_security_group" "nsg" {
  name                = "azurevm1_nsg"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_network_security_rule" "ssh_rule" {
  name                        = "AllowSSH"
  priority                    = 100
  direction                   = "Inbound"
  access                      = "Allow"
  protocol                    = "Tcp"
  source_port_range           = "*"
  destination_port_range      = "22"
  source_address_prefix       = "*"
  destination_address_prefix  = "*"
  resource_group_name         = azurerm_resource_group.rg.name
  network_security_group_name = azurerm_network_security_group.nsg.name
}

resource "azurerm_subnet_network_security_group_association" "nsg_association" {
  subnet_id                 = azurerm_subnet.subnet.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}
```

```
resource "azurerm_linux_virtual_machine" "azurevm1" {
  name                = "azurevm1"
  resource_group_name = azurerm_resource_group.rg.name
  location            = azurerm_resource_group.rg.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  network_interface_ids = [
    azurerm_network_interface.nic.id,
  ]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file(var.ssh_public_key)
  }

  os_disk {
    name                 = "azurevm1_disk1"
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
    disk_size_gb         = 30
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "ubuntu-24_04-lts"
    sku       = "server"
    version   = "latest"
  }
}
```

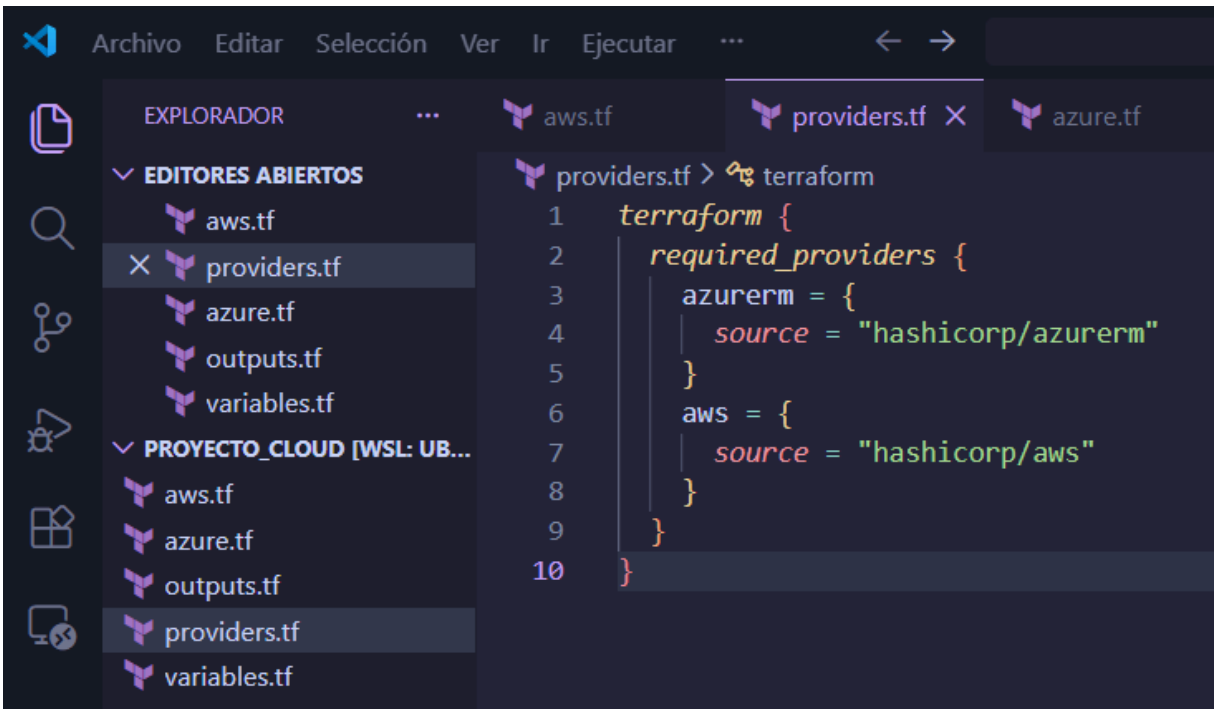
Seguidamente configuraremos el archivo de las variables, variables.tf para que no hayan errores en los archivos de configuración.

```
aws.tf    azure.tf    outputs.tf    variables.tf
variables.tf > variable "ssh_public_key"
1   # AWS
2   variable "aws_region" {
3     default = "us-east-1"
4   }
5
6   # Azure
7   variable "azurerm_region" {
8     default = "westus2"
9   }
10
11  variable "ssh_public_key" {
12    description = "SSH Public Key"
13  }
```

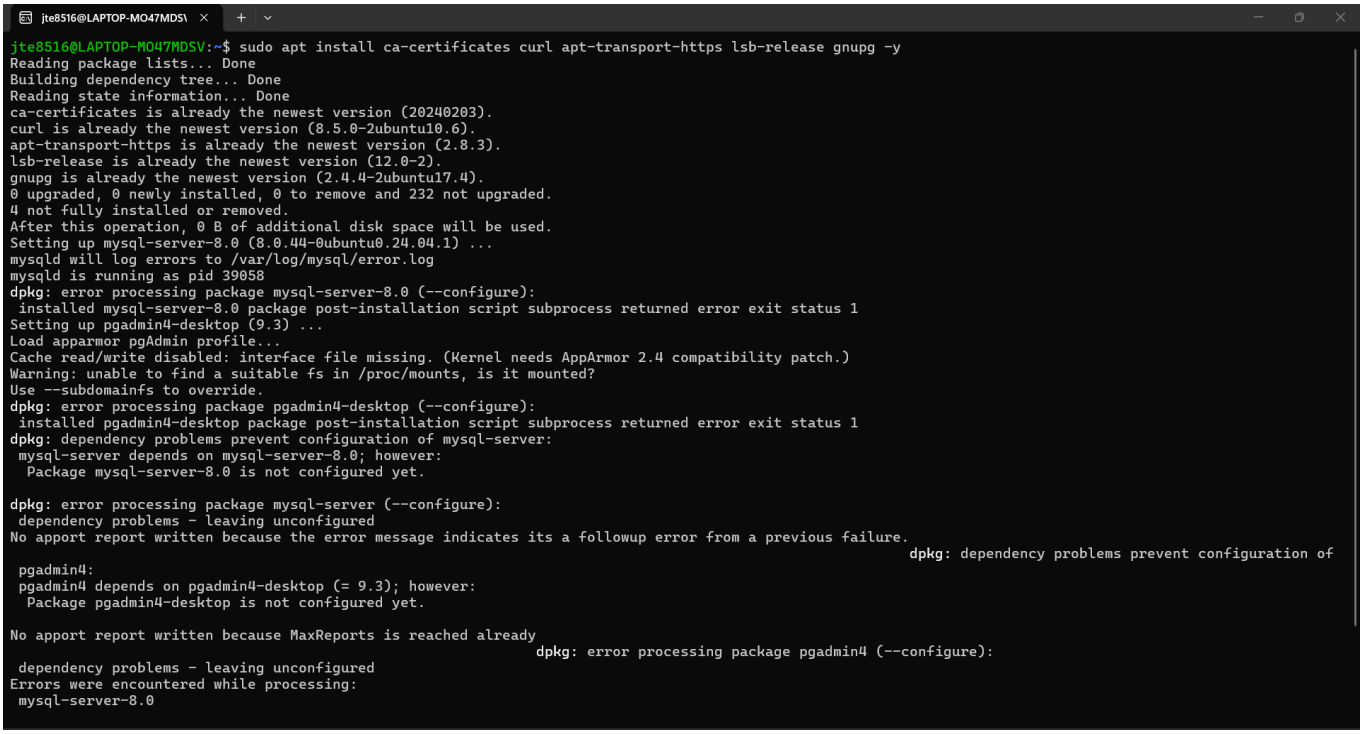
Configuramos el archivo outputs.tf que este sirve para definir las salidas que queremos que Terraform muestre al final de la ejecución de terraform apply.

```
Ir   Ejecutar   ···   ← →   proyecto_cloud [WSL: Ubuntu]
aws.tf    azure.tf    outputs.tf    variables.tf
outputs.tf > output "azure_private_ip"
1   # Outputs para AWS
2   output "aws_public_ip" {
3     value       = aws_instance.awsvm1.public_ip
4     description = "IP pública de la instancia AWS"
5   }
6
7   output "aws_private_ip" {
8     value       = aws_instance.awsvm1.private_ip
9     description = "IP privada de la instancia AWS"
10  }
11
12  # Outputs para Azure
13  output "azure_public_ip" {
14    value       = azurerm_public_ip.publicip.ip_address
15    description = "IP pública de la VM en Azure"
16  }
17
18  output "azure_private_ip" {
19    value       = azurerm_linux_virtual_machine.azurevm1.private_ip_address
20    description = "IP privada de la VM en Azure"
21  }
```

Para finalizar crearemos el archivo [provider.tf](), que en este archivo configuraremos lo que serán nuestros proveedores.



Instalamos las herramientas para poder conectarnos con azure y de esta forma al hacer el terraform apply se subiran los recursos que tocan tanto en azure como en Amazon Web Service.



```
jte8516@LAPTOP-MO47MDSV:~$ curl -sL https://packages.microsoft.com/keys/microsoft.asc | sudo gpg --dearmor -o /usr/share/keyrings/microsoft-archive-keyring.
gpg
jte8516@LAPTOP-MO47MDSV:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/microsoft-archive-keyring.gpg] https://packages.micros
oft.com/repos/azure-cli/ $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/azure-cli.list
deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft-archive-keyring.gpg] https://packages.microsoft.com/repos/azure-cli/ noble main
jte8516@LAPTOP-MO47MDSV:~$
```

```
jte8516@LAPTOP-MO47MDSV:~$ sudo apt install azure-cli -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  azure-cli
0 upgraded, 1 newly installed, 0 to remove and 232 not upgraded.
4 not fully installed or removed.
Need to get 54.1 MB of archives.
After this operation, 608 MB of additional disk space will be used.
Get:1 https://packages.microsoft.com/repos/azure-cli noble/main amd64 azure-cli amd64 2.83.0-1~noble [54.1 MB]
Fetched 54.1 MB in 13s (4215 kB/s)
Selecting previously unselected package azure-cli.
(Reading database ... 105673 files and directories currently installed.)
Preparing to unpack .../azure-cli_2.83.0-1~noble_amd64.deb ...
Unpacking azure-cli (2.83.0-1~noble) ...

Progress: [  8%] [##########.....................................................................................]
```

Para iniciar sesión utilizamos lo siguiente y entramos dentro de la URL y ponemos el código que nos aparece:

```
^Cjte8516@LAPTOP-MO47MDSV:~$ ^C
jte8516@LAPTOP-MO47MDSV:~$ az login --use-device-code
To sign in, use a web browser to open the page https://microsoft.com/devicelogin and enter the code GU9JJX6ZA to authenticate.
sudo apt install azure-cli -y
```

Una vez iniciado sesión nos saldrá lo siguiente conforme hemos iniciado sesión:

```
[Tenant and subscription selection]

No     Subscription name    Subscription ID                       Tenant
-----  -------------------  ------------------------------------  --------------
[1] *  Azure for Students   c434d080-e2db-4441-add1-e24ea1abc27b  INS Provençana

The default is marked with an *; the default tenant is 'INS Provençana' and subscription is 'Azure for Students' (c434d080-e2db-4441-add1-e24ea1abc27b).

Select a subscription and tenant (Type a number or Enter for no changes): 1
Invalid selection.
Select a subscription and tenant (Type a number or Enter for no changes):

Tenant: INS Provençana
Subscription: Azure for Students (c434d080-e2db-4441-add1-e24ea1abc27b)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://g
o.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

jte8516@LAPTOP-MO47MDSV:~$
```

Una vez ya estamos conectados con aws y azure, hacemos terraform init para iniciar la conexión.

Ver Ir Ejecutar ··· proyecto_cloud [WSL: Ubuntu

aws.tf providers.tf azure.tf outputs.tf variables.tf

azure.tf > resource "azurerm_subnet" "subnet"

```
3   }
4
5   resource "azurerm_resource_group" "rg" {
6     name     = "azurevm1_rg"
7     location = var.azurerm_region
8   }
9
10  resource "azurerm_virtual_network" "vnet" {
11    name                = "azurevm1_vnet"
12    address_space       = ["10.0.0.0/16"]
13    location            = azurerm_resource_group.rg.location
14    resource_group_name = azurerm_resource_group.rg.name
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

```
jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/azurerm...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/azurerm v4.61.0...
- Installed hashicorp/azurerm v4.61.0 (signed by HashiCorp)
- Installing hashicorp/aws v6.33.0...
- Installed hashicorp/aws v6.33.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$
```

Seguidamente realizamos el terraform plan para ver cuál será el resultado que nos dará todo el código:

```
jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$ terraform plan
      + name                                      = "azurevm1_subnet"
      + private_endpoint_network_policies         = "Disabled"
      + private_link_service_network_policies_enabled = true
      + resource_group_name                       = "azurevm1_rg"
      + virtual_network_name                      = "azurevm1_vnet"
    }

  # azurerm_subnet_network_security_group_association.nsg_association will be created
  + resource "azurerm_subnet_network_security_group_association" "nsg_association" {
      + id                        = (known after apply)
      + network_security_group_id = (known after apply)
      + subnet_id                 = (known after apply)
    }

  # azurerm_virtual_network.vnet will be created
  + resource "azurerm_virtual_network" "vnet" {
      + address_space                 = [
          + "10.0.0.0/16",
        ]
      + dns_servers                   = (known after apply)
      + guid                          = (known after apply)
      + id                            = (known after apply)
      + location                      = "westeurope"
      + name                          = "azurevm1_vnet"
      + private_endpoint_vnet_policies = "Disabled"
      + resource_group_name           = "azurevm1_rg"
      + subnet                        = (known after apply)
    }

Plan: 9 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ aws_public_ip    = "107.21.161.61" -> "54.87.42.64"
  + azure_private_ip = (known after apply)
  + azure_public_ip  = (known after apply)


Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$
```

Y una vez ya sabemos que se nos aplicará, confirmaremos los cambios para que se aplique todo realizando un terraform apply:

```
jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$ terraform apply

  Error: creating Virtual Network (Subscription: "c434d080-e2db-4441-add1-e24ea1abc27b"
  Resource Group Name: "azurevm1_rg"
  Virtual Network Name: "azurevm1_vnet"): performing CreateOrUpdate: unexpected status 403 (403 Forbidden) with error: RequestDisallowedByAzure: Resource 'azure
vm1_vnet' was disallowed by Azure: This policy maintains a set of best available regions where your subscription can deploy resources. The objective of this pol
icy is to ensure that your subscription has full access to Azure services with optimal performance. Should you need additional or different regions, contact sup
port..

    with azurerm_virtual_network.vnet,
    on azure.tf line 6, in resource "azurerm_virtual_network" "vnet":
     6: resource "azurerm_virtual_network" "vnet" {


  Error: updating Public I P Address (Subscription: "c434d080-e2db-4441-add1-e24ea1abc27b"
  Resource Group Name: "azurevm1_rg"
  Public I P Addresses Name: "azurevm1-publicip"): performing CreateOrUpdate: unexpected status 403 (403 Forbidden) with error: RequestDisallowedByAzure: Resour
ce 'azurevm1-publicip' was disallowed by Azure: This policy maintains a set of best available regions where your subscription can deploy resources. The objectiv
e of this policy is to ensure that your subscription has full access to Azure services with optimal performance. Should you need additional or different regions
, contact support..

    with azurerm_public_ip.publicip,
    on azure.tf line 20, in resource "azurerm_public_ip" "publicip":
    20: resource "azurerm_public_ip" "publicip" {


  Error: creating Network Security Group (Subscription: "c434d080-e2db-4441-add1-e24ea1abc27b"
  Resource Group Name: "azurevm1_rg"
  Network Security Group Name: "azurevm1_nsg"): performing CreateOrUpdate: unexpected status 403 (403 Forbidden) with error: RequestDisallowedByAzure: Resource
'azurevm1_nsg' was disallowed by Azure: This policy maintains a set of best available regions where your subscription can deploy resources. The objective of thi
s policy is to ensure that your subscription has full access to Azure services with optimal performance. Should you need additional or different regions, contac
t support..

    with azurerm_network_security_group.nsg,
    on azure.tf line 41, in resource "azurerm_network_security_group" "nsg":
    41: resource "azurerm_network_security_group" "nsg" {

jte8516@LAPTOP-MO47MDSV:~/proyecto_cloud$
```

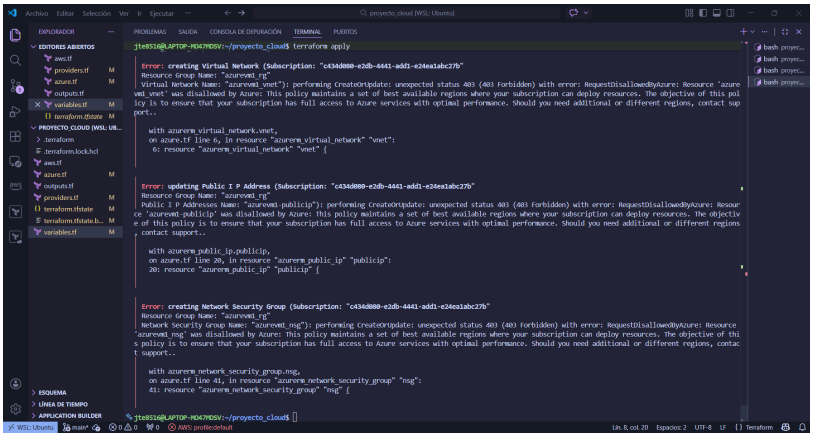
3. DIFICULTADES

Para empezar, nos ha costado mucho conseguir una cuenta de Azure, porque nosotros tenemos una cuenta corporativa del Instituto con cuenta de Azure, pero no teníamos saldo en ella y no nos dejaba renovar el saldo.

Al principio de todo nos costó mucho conseguir información sobre azure, y también estamos acostumbrados a trabajar sobre Amazon Web Service. Entonces para nosotros nos ha sido más complicado trabajar sobre Azure ya que hay bastantes cosas que son muy diferentes.

Al hacer el terraform init, no hemos tenido ningún tipo de problema con AWS pero sin embargo con Azure hemos tenido varios problemas, como por ejemplo errores con la zona, y el error que más nos ha molestado y que no hemos podido solventar ha sido el error de la verificación/autenticación, al hacer el terraform apply, azure nos daba un error de verificación/autenticación, que aun así identificandonos conforme era nuestra la cuenta, no nos dejaba tampoco.

Lo único que nos permite Azure es crear un grupo de recursos, pero por lo que se ve, al tener una cuenta de estudiantes, las políticas de dentro no nos permite crear nada más.



4. TRABAJO FUTURO

Lo que nosotros haríamos de cara a un futuro o si tuviésemos más tiempo sería lo siguiente:

- **Implementación de CI/CD con GitHub Actions:** Automatizamos todo el proceso de despliegue. De esta forma, cada vez que hiciéramos un "push" al repositorio de GitHub, las herramientas de integración continua ejecutarán el terraform apply de forma automática, sin tener que hacerlo manualmente desde nuestra terminal.
- **Modularización del código:** Actualmente tenemos archivos generales para cada proveedor, pero lo ideal sería crear módulos de Terraform. Esto nos permitiría reutilizar el código de la red o de las máquinas virtuales para otros proyectos simplemente llamando al módulo, haciendo que el código sea mucho más limpio y profesional.
- **Uso de un Backend Remoto:** En este proyecto el archivo de estado (terraform.tfstate) se guarda en local. Para un entorno de trabajo real, configuraremos un backend remoto en un bucket de S3 (AWS) o en un Storage Account (Azure). Esto permitiría que varias personas trabajaran en el mismo proyecto a la vez sin sobrescribir los cambios de los demás.
- **Alta disponibilidad y balanceo de carga:** En lugar de una única instancia para el WordPress, desplegaríamos un balanceador de carga (ELB en AWS o Load Balancer en Azure) y un grupo de auto-escalado. Así, si una máquina falla, el servicio seguiría funcionando y podría escalar automáticamente según el tráfico.
- **Conectividad Multi-cloud:** Configuraremos una conexión VPN Site-to-Site o un túnel entre AWS y Azure. De este modo, los recursos de ambas nubes podrían comunicarse entre sí a través de sus IPs privadas de forma segura, como si estuvieran en la misma red local.

- **Bases de datos gestionadas:** En lugar de instalar la base de datos dentro de la misma máquina virtual, usaríamos servicios gestionados como AWS RDS o Azure Database for MySQL. Esto mejoraría mucho la seguridad, el rendimiento y nos facilitaría las copias de seguridad del WordPress.